

Aim:

You are the chief archivist at Nalanda Mahavihara, the legendary ancient Indian university renowned for its vast collection of manuscripts. Palm-leaf manuscripts and birch-bark scrolls are stored in meticulously catalogued order using archive IDs.

The Rajguru (Royal Priest) requests a specific manuscript for a sacred Vedic ritual. Multiple copies of the same manuscript may exist, but tradition dictates that the priest must always receive the oldest preserved copy, which corresponds to the first occurrence of the archive ID in the catalog.

Your challenge is to locate this first occurrence efficiently. A linear search through the sprawling manuscript vault would take too long and result in a Time Limit Exceeded (TLE) verdict. Instead, you must apply the power of Binary Search to swiftly find the earliest preserved copy.

Algorithm Design

The program solves the problem of finding the first occurrence of a given Archive ID in a sorted list of manuscript IDs.

1. Input Phase

- Read number of scrolls n .
- Read sorted archive IDs into an array.
- Read the target archive ID to search.

2. Binary Search Phase

Initialize $low = 0$, $high = n - 1$, $result = -1$.

While $low \leq high$:

- Compute $mid = low + (high - low) / 2$.
- If $arr[mid] == target$:
- Record $result = mid$.
- Move $high = mid - 1$ to continue searching left for earlier occurrence.
- Else if $arr[mid] < target$: move $right(low = mid + 1)$.
- Else: move $left(high = mid - 1)$.

Return result.

3. Output Phase

- If $result \neq -1$, print the index of the first occurrence.
- Otherwise, print "Scroll not found" message.

Algorithm Analysis**1. Time Complexity:**

- Best Case: $O(\log n)$ (target found quickly).
- Worst Case: $O(\log n)$ (still logarithmic, even if target is absent).
- Average Case: $O(\log n)$.

2. Space Complexity:

- $O(1)$ (no extra memory used, iterative approach).

3. Suitability:

- Efficient for large archives (millions of IDs).
- Guarantees finding the first occurrence even with duplicates.

Input Format:

- The first line contains an integer n , representing the number of scrolls in the archive.
- The second line contains n space-separated sorted integers representing the archive IDs.
- The third line contains an integer $target$, representing the archive ID to be searched.

Output Format:

- If the archive ID is found, print:

Scroll found at position: <index>

where, index is the position of the first occurrence of the archive ID.

- Otherwise, print:

Scroll not found

Note:

- Skeleton code is provided. Complete the missing logic in **findFirstOccurrence()** to find the first occurrence of the target archive ID using Binary Search.

Source Code:

firstOccurrenceFinding.c

```
#include <stdio.h>

// finding first occurrence of Archive ID
int findFirstOccurrence(int arr[], int n, int target) {
    int index=-1;
    for(int i=0;i<n;i++){
        if(arr[i]==target){
            index=i;
            break;
        }
    }
    return index;
}

int main() {
    int n, target;
    printf("scrolls: ");
    scanf("%d", &n);
    int archive[n];
    printf("sorted Archive IDs:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &archive[i]);
    }
    printf("Archive ID: ");
    scanf("%d", &target);
    int index = findFirstOccurrence(archive, n, target);
    if (index != -1) {
        printf("Scroll found at position: %d\n", index);
    } else {
        printf("Scroll not found\n");
    }
    return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
scrolls:
1
sorted Archive IDs:
100
Archive ID:
100
Scroll found at position: 0

Test Case - 2
User Output
scrolls:
5
sorted Archive IDs:
100 200 200 200 300
Archive ID:
200
Scroll found at position: 1

Test Case - 3
User Output
scrolls:
4
sorted Archive IDs:
50 100 200 300
Archive ID:
400
Scroll not found